

P o l y m o r p h i s m

Polymorphism

Principle

Vary by type, not by if/else

DEFINITION

When behaviour varies by type, assign it through polymorphic operations rather than conditionals on type.

When behaviour depends on type, you have two options: branch on a type tag everywhere it matters, or define one operation and let each type implement it. GRASP Polymorphism says choose the second — assign the varying behaviour to the types themselves through a shared interface, so the runtime dispatches to the right variant and the conditionals vanish.

WHY IT MATTERS

Type-switch ladders are duplicated at every call site and must be edited for each new type — easy to miss one, and the compiler won't warn you. Moving each variant behind a common operation centralises it, makes adding a type an additive change (Open/Closed), and lets clients work against the abstraction without knowing the concrete type at all.

— How it works

Interface	The common operation every variant implements — the single seam clients call.
Variants	The type-specific implementations, one per case that used to be a switch branch.
Client	Calls the operation against the interface, with no knowledge of which concrete type runs.

HOW TO APPLY

1. Find the repeated switch / if ladder on a type or kind field.
2. Define a common operation that captures what varies (e.g. area(), price()).
3. Give each type its own implementation of that operation.
4. Replace the conditionals with a single polymorphic call.

LITMUS TEST

Do you switch on a type tag or use instanceof in more than one place? Each such ladder is a candidate to replace with a polymorphic operation.

— Smell → fix

A switch on a kind field — every new shape forces an edit here and everywhere it is used:

```
function area(s: Shape) {  
  switch (s.kind) { // edit for every new type  
    case 'circle': return Math.PI * s.r * s.r  
    case 'square': return s.side * s.side  
  }  
}
```

↓ DISPATCH ON THE TYPE

```
interface Shape { area(): number }  
class Circle implements Shape {  
  area() { return Math.PI * this.r * this.r }  
}  
// client calls shape.area() — runtime picks the variant
```

Each shape (Circle, Square, ...) owns its formula; the client never checks types. A new shape is a new class — no existing code changes (Open/Closed).

USE WHEN

- ✓ switch / if ladders on a type field
- ✓ Pluggable variation by subtype

AVOID WHEN

- ▲ A single, stable behaviour — polymorphism is needless

— Trade-offs & pitfalls

PROS

- + Open to new types (OCP)
- + Removes conditionals

CONS

- More classes
- Indirection if overused

COMMON PITFALLS

- ▲ Polymorphism for behaviour that never varies — needless classes and indirection.
- ▲ Variants that aren't substitutable (break LSP) — the dispatch then lies to its callers.
- ▲ Scattering one decision across many files when a simple, local conditional was clearer.

WHERE YOU SEE IT

- `shape.area()` across `Circle` / `Square` / `Triangle` instead of `switch(shape.kind)`.
- GoF Strategy and State — swapping algorithms or state-specific behaviour by type.
- A renderer / serializer with one implementation per format, selected by the object's class.

SEE ALSO

OCP	polymorphic variants are what you extend without editing callers
Strategy	a GoF pattern that applies polymorphism to swap algorithms
State	applies polymorphism to swap behaviour as state changes
LSP	variants must be substitutable for the interface, or dispatch breaks

— Interview & sources

Polymorphism (GRASP) vs Strategy (GoF)?

GRASP Polymorphism is the principle (replace type checks with subtype dispatch). Strategy and State are patterns that apply it to swap algorithms or states.

Is every conditional bad?

No — only repeated branching on a type or kind. Branching on values, validation, or a one-off check is fine. Polymorphism pays off when the same type-switch recurs and grows.

Downside of polymorphism?

More classes and indirection; logic spreads across types, so one variant is easy to read but the whole set is harder to survey. Overusing it for stable, single behaviour is needless.

How does polymorphism enable Open/Closed?

New behaviour arrives as a new class implementing the existing interface — callers don't change. The type-switch alternative forces an edit in every branch for each new case, which is exactly what OCP tells you to avoid.

REMEMBER

When behaviour varies by type, let the type decide — not an if or switch.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Polymorphism

GoF — "Design Patterns" (1994): Strategy and State